AsyncWebRL: Async RL for Visual Web Agents

H. Bai, R. Yang, C. Ye, S. Whitehead, A. Kumar, T. Zhang

Paper version: arXiv 2606.05597v3 · CUA Radar Reel poster, not made or endorsed by the authors

Problem

> *idle GPUs in synchronous RL, and trajectories that use more steps and tokens than necessary*

Async RL exists for text, not visual multi-step agents; and the GRPO factor $1/|\tau_i|$ quietly rewards verbose failures.

Two inefficiencies

1. **Idle GPUs.** Sync RL waits on the slowest trajectory.
2. **Screenshot flood.** Per-step frames spill the shared store to disk.
3. **Verbose failures.** $1/|\tau_i|$ down-weights long failures → bloat.

Evaluation setup

WebGym · 290k tasks | OOD test · 1,167 tasks | Qwen3-VL-8B Instruct | Qwen3-VL-8B Thinking | Horizons 10/20/30 | Binary GPT-4o reward | Single seed | 2×8 B200 GPUs

- **Online eval only:** success on held-out OOD websites; no offline step metric.
- **Baselines:** published sync WebGym + like-for-like async RAFT++.

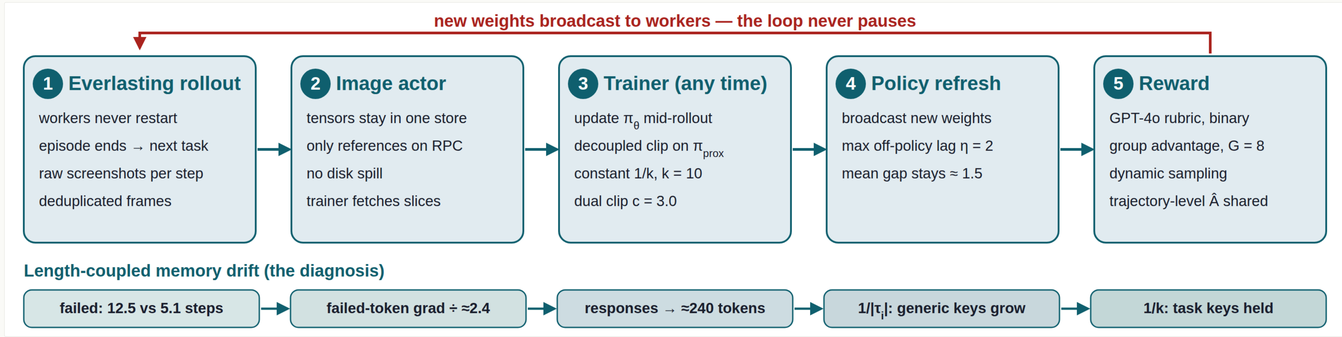
Our read (CUA Radar)

61	★★	transfer
score of 100	band	evidence level

- **Reports:** online eval; held-out OOD sites; strong baseline; matched horizons; cost; ablation.
- **Flags:** single seed; GPT-4o rubric reward unvalidated; code link unclear.
- Sub-pattern K20. Authors' numbers, not rerun.

Method: overlap everything, then fix the loss

- **Sees / acts:** prev screenshot + Memory/Progress prompt in; coordinate click / type / scroll out.
- **System:** everlasting rollout pool + reference-only image actor; rollout, update, refresh overlap.
- **Signal:** binary GPT-4o rubric reward → group advantage (G = 8); dynamic sampling, no KL.



Our redraw (not the paper's figure): the always-on loop and the two loss fixes.

$$J(\theta) = \mathbb{E}\left[\frac{1}{G \cdot k}\sum_{i}\sum_{j}\sum_{t}\min\left(\frac{\pi_\theta}{\pi_{\text{behave}}}\hat{A}_i,\ \text{clip}\left(\frac{\pi_\theta}{\pi_{\text{prox}}},\ 1-\varepsilon,\ 1+\varepsilon\right)\hat{A}_i\right)\right]$$

Decoupled clip centered on π_{prox}; constant 1/k replaces $1/|\tau_i|$ ($k = 10$).

Peak success rate (%) on WebGym OOD test split

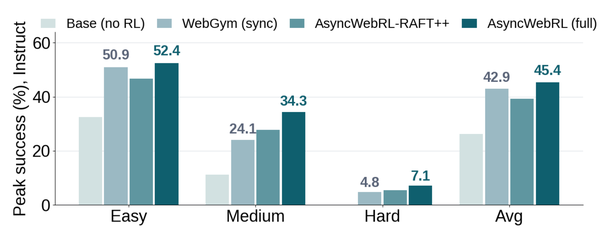


Table 1, redrawn: Qwen3-VL-8B-Instruct, single seed, peak online success (%).

Harder slices: +42% Medium, +48% Hard (relative).

SO WHAT → *Thinking: full 44.4 vs RAFT++ 40.5 Avg.*

Diagnosis: Memory schema signatures

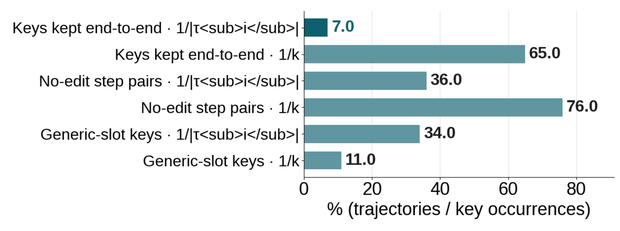


Table 2, redrawn: Instruct, additive prompt, first 30 steps per trajectory.

SO WHAT → *1/k holds a small task-anchored key set; under $1/|\tau_i|$ the schema rewrites generic keys every step. Test reward is essentially tied between the two losses — only length and schema change. Under $1/|\tau_i|$ responses grow to ≈240 tokens/step; scaled horizons 20/40/60 amplify the drift.*

Headline Numbers

45.4%
WebGym OOD success
vs 42.9% sync WebGym

2.4–2.9× end-to-end speedup

1.80× per-step, 1/k fix

≈3,100 training trajs / hour

The paper's own controls

CONTROL	RESULT	CONTEXT
Coupled vs decoupled	coupled ≈ 2× clip rate	kept in all runs
Async like-for-like	RAFT++ 39.3 vs 42.9	same loss family
Off-policy gap, η cap 2	mean ≈ 1.5, max ≈ 2.0	GPUs stay busy
Prompt-side fix	compressive prompt fails	edits above Base

Takeaway

Async system + decoupled clip + 1/k fix: 2.4–2.9× throughput, 45.4% vs 42.9% OOD, single seed.

failed 12.5 vs 5.1 steps | responses ≈240 tok/step | 2 nodes × 8 B200

Scan to Read

Paper | Code